Rapport scientifique et audit

NGeGenBrain v3_0

Réentraînement anglais et reconstruction multi-facettes d'un tiny causal language model

Dossier expérimental NGeGenBrain

28 octobre 2026

Version logicielle	NGeGenBrain v3_0 (3.0.0)
Modèle produit	NG-Recomposed-TinyLM-v3_0
Campagne	NG-V3_0-EXP-01 / EXP-02
Environnement	Python 3.13.5, PyTorch 2.10.0+cpu, CPU
Statut global	Tiny LM causal exécutable ; qualité conversationnelle non validée

Table des matières

1 Description **3**

1.1 Questions expérimentales . . . 3

2 Architecture **3**

2.1 Modèle v3 de référence . . . 3

2.2 Architecture multi-facettes v3_0 . . . 3

2.3 Décodage causal . . . 4

3 Dataset anglais et protocole d'intégrité **4**

4 Pipeline complet **4**

5 Commandes de reproduction **5**

6 Résultats **5**

6.1 Résultats globaux sur le test final . . . 5

6.2 Performances par facette . . . 5

6.3 Observations des représentations . . . 6

6.4 Interventions causales . . . 6

6.5 Génération . . . 6

7 Interprétation des résultats **6**

8 Conséquences théoriques, conceptuelles et opérationnelles **7**

8.1 Théoriques . . . 7

8.2 Conceptuelles 7
8.3 Opérationnelles 7

9 Audit des restrictions de code **7**

10 Limites et contre-exemples **7**

11 Conclusions **8**

12 Statut du modèle **8**

13 Traçabilité **8**

14 Reproductibilité **9**

1 Description

Ce rapport documente deux expériences liées sur le tiny language model autonome produit précédemment par NGeGenBrain v3. La première construit un dataset anglais de documents originaux, reprend l'entraînement à partir des poids v3, et évalue le résultat sur un test réservé. La seconde modifie la procédure de construction du modèle : au lieu de n'observer que la prédiction du prochain octet, elle traite plusieurs facettes observables d'un même corpus et les combine dans un modèle à cœur causal partagé, routeur de facettes et adaptateurs spécialisés.

L'objectif n'est pas d'affirmer une réussite à l'avance. Le statut scientifique est décidé sur les mesures observées. Le modèle source de l'expérience est le modèle de 41 728 paramètres livré dans `model_produit_NG_v3.zip`. Le nouveau modèle contient 54 536 paramètres. Aucune inférence du modèle enseignant n'est requise par le modèle produit lors de l'exécution.

1.1 Questions expérimentales

1. Le réentraînement avec un dataset anglais documenté améliore-t-il les prédictions de langage par rapport aux poids v3 non réentraînés et à une baseline statistique simple ?
2. Une construction multi-facettes apprend-elle une classification de documents au-dessus du hasard et conserve-t-elle une amélioration de la prédiction des octets sur un test inédit ?
3. Le code ancien comporte-t-il des choix pouvant artificiellement restreindre les résultats, indépendamment de l'intention de ses auteurs ?
4. Le modèle produit peut-il être chargé depuis une archive et utilisé par l'interface/API locale ?

2 Architecture

2.1 Modèle v3 de référence

Le modèle historique est un modèle causal byte-level : vocabulaire de 256 valeurs, embedding, une couche GRU à état caché de 64, normalisation et tête de prédiction du prochain octet. Il a 41 728 paramètres. Le tokeniseur opère sur les octets UTF-8 et non sur des sous-mots. Le modèle peut donc être un language model neuronal réel sans que cela garantisse une génération grammaticalement correcte ou un comportement conversationnel.

2.2 Architecture multi-facettes v3_0

Le modèle final comporte 54 536 paramètres : embedding partagé, GRU à une couche (dimension cachée 64), normalisation, tête de prochain octet, routeur à huit classes et huit adaptateurs bas-rang (rang 4) avec correction de biais par facette. Les huit facettes sont `explanation`, `procedure`, `dialogue`, `narrative`, `question_answer`, `contrastive`, `definition` et `scientific_reasoning`. Les noms sont les catégories du benchmark ; ils ne doivent pas être interprétés comme des concepts internes prouvés.

Les poids compatibles du modèle v3 (embedding, GRU, normalisation et tête de prédiction) sont chargés comme initialisation. Les poids nouvellement ajoutés du routeur et des adaptateurs sont initialisés puis entraînés. La perte de sélection conjointe est :

$$\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{next-byte}} + 0.8\,\mathcal{L}_{\text{facet}}.$$

Le checkpoint est choisi à partir de la validation, à l'étape 900, et non sur le score du test final.

2.3 Décodage causal

Le code v3_0 effectue un préremplissage du prompt puis fournit un seul nouvel octet par étape tout en propageant l'état récurrent. Cette modification évite de réinjecter tout le prompt à chaque étape en même temps que l'état caché passé. Le nombre de nouveaux octets est contrôlé par l'appelant ; les limites physiques de mémoire et de temps demeurent présentes. Le champ `context_window=96` de la configuration correspond à la longueur de séquence utilisée pour l'entraînement ; il n'impose pas un plafond global de longueur dans le décodeur.

3 Dataset anglais et protocole d'intégrité

Le dataset `NG-English-Multifacet-TinyLM-v3_0-final2` contient 144 documents anglais rédigés spécialement pour ce benchmark contrôlé, sans texte externe téléchargé. Il comprend 112 documents d'entraînement, 16 de validation et 16 de test, soit 14 documents d'entraînement et 2 de validation / test pour chacune des huit facettes. Le nombre d'octets/caractères sérialisés est de 50 756 pour l'entraînement, 7 912 pour la validation et 7 743 pour le test selon le manifeste ; les mesures finales scorent 7 727 positions de prochain octet après exclusion des préfixes.

Les essais pilotes précédents ont été consultés pendant le développement. En conséquence, tous les exemples précédemment vus ont été relégués dans le split train/dev ; la validation et le test final2 utilisent 32 documents thématiques nouveaux, non utilisés au cours de ces essais. Le test final contient 16 documents, deux par facette. Le manifeste possède l'empreinte SHA-256 :

```
d9919e0713bd33863dd08b30dd2b5d7974da6892e2bb2da2f6485877ca0b9281
```

Le corpus est contrôlé et relativement petit ; les résultats ne sont pas des scores sur un benchmark linguistique standard ni une approximation non biaisée de la performance sur le Web. Le principal avantage de cette construction est la provenance claire et le contrôle des splits ; la principale limite est sa distribution documentaire synthétique.

4 Pipeline complet

1. Audit de l'ancien code d'entraînement, de la génération, de la configuration et du chargement local.
2. Construction des documents anglais et mise à jour du manifeste de séparation.
3. Vérification de l'unicité des identifiants, des empreintes, et de la séparation des textes entre splits.
4. Mesure v3 de référence sur le nouveau test final.
5. Entraînement v3_0 depuis les poids v3, avec prédiction du prochain octet et classification des facettes.
6. Enregistrement périodique des pertes train/validation et sélection du checkpoint via la perte conjointe de validation.
7. Évaluation finale sur les 16 documents de test réservés après le gel du checkpoint.
8. Comparaison avec une baseline trigramme d'octets avec backoff bigramme/unigramme et lissage additif, estimée sur le split train seulement.
9. Observations multi-vues : pertes par facette, entropie prédictive, résumé statistique des activations, rang effectif, ablations des 64 dimensions cachées, et génération contrôlée par prompts de facette.
10. Test de chargement du modèle depuis dossier puis ZIP par le registre local, suivi de tests via l'API HTTP de l'interface.

5 Commandes de reproduction

À partir de la racine du dépôt dans un environnement Python avec PyTorch installé :

```
python -m pip install -e ".[torch]"
python ngbrain.py produce-multifacet-tinylm \\
  --dataset research/english_dataset_v3_0_final2 \\
  --base-model references/NG-Recomposed-TinyLM-v3/model \\
  --output model_produit_NG_v3_0 --steps 900 \\
  --batch-size 12 --seq-len 96 --learning-rate 0.001 \\
  --facet-weight 0.8 --rank 4 --seed 20261028
python -m unittest discover -s tests -v
python ngbrain.py --workspace work_web serve \\
  --host 127.0.0.1 --port 8766
```

Le bundle de suivi conserve les poids originaux de référence ainsi que le dataset et les scripts. Le modèle produit fourni peut être utilisé directement sans réexécution de l'entraînement. Une reproductibilité bit-à-bit entre matériels et versions de bibliothèques n'est pas garantie ; la configuration, la graine, les traces et les empreintes sont archivées.

6 Résultats

6.1 Résultats globaux sur le test final

Le test final comprend 7 727 octets de cibles après exclusion des préfixes contextuels. Le modèle v3_0 a été évalué après choix du checkpoint, sans tuning sur ces documents.

Mesure	V3 original	Baseline trigramme	V3_0 multi-facettes
NLL (nats/octet)	3.1392	1.9988	**1.8530**
Perplexité par octet	23.085	7.380	**6.379**
Exactitude prochain octet top-1	21.81%	41.59%	**48.16%**
Paramètres	41 728	–	54 536
Exactitude de facette	–	–	62.5% (10/16)

La NLL de v3_0 baisse d'environ 41.0% par rapport au modèle v3 original sur ce test. La perplexité et l'exactitude top-1 surpassent aussi la baseline trigramme. Le score de classification de facette est 10/16 documents ; avec seulement deux documents test par classe, l'intervalle d'incertitude est large et le résultat ne prouve pas la robustesse du routeur.

6.2 Performances par facette

Facette	NLL (nats/octet)	Perplexité/octet
Explication	2.212	9.132
Procédure	1.780	5.928
Dialogue	1.688	5.410
Narration	1.921	6.827
Question-réponse	1.780	5.929
Contraste	1.825	6.203
Définition	1.792	6.004
Raisonnement scientifique	1.803	6.068

L'explication est la catégorie la plus difficile du test par NLL. Les chiffres montrent une variation mesurable, mais la quantité d'exemples par facette ne suffit pas à établir une hiérarchie générale des capacités du modèle.

6.3 Observations des représentations

Les représentations cachées ont été résumées sur des préfixes des documents test sans fournir les labels au modèle pour sa prédiction du prochain octet. Le rang effectif estimé varie selon les facettes ; il est de l'ordre de 6 à 10 sur une dimension cachée de 64, selon la facette et le calcul de participation. Il s'agit d'une mesure descriptive de dispersion des activations, pas d'une preuve qu'une dimension code un concept discret.

6.4 Interventions causales

L'expérience d'ablation a mis à zéro une dimension d'état caché après chaque étape récurrente, puis propagé l'état modifié. Les 64 dimensions ont été testées sur les premières 64 positions de 16 documents de validation, soit 1 024 cibles scorées. Les cinq dimensions les plus sensibles selon la hausse moyenne de NLL sont : unité 62 (+0.609 nats/octet), unité 17 (+0.492), unité 12 (+0.463), unité 44 (+0.422), unité 54 (+0.415).

Cette expérience démontre que la prédiction est sensible à ces interventions spécifiques dans ces séquences. Elle ne permet pas de baptiser sémantiquement ces unités, et ne prouve pas leur nécessité hors distribution. D'autres interventions (patching entre exemples, permutation, ablation de groupes et tests de sauvetage) seraient nécessaires avant des conclusions plus fortes.

6.5 Génération

Des prompts ont été testés dans chacune des huit facettes. Les générations font apparaître des fragments liés au domaine et à la structure des textes, mais restent fortement bruitées, souvent mal formées et peu cohérentes à l'échelle de 120 caractères. Par exemple, la suite de « A rain garden works because » démarre par « depearng under that may compare on a spection... ». Ce contre-exemple invalide toute affirmation que le produit fonctionne déjà comme assistant conversationnel robuste. Il ne invalide pas sa qualification technique de modèle neuronal causal entraîné à prédire le prochain octet.

7 Interprétation des résultats

Les données soutiennent trois conclusions bornées. Premièrement, le produit v3_0 a été effectivement entraîné et sa prédiction du prochain octet s'est améliorée sur le test final réservé. Deuxièmement, le routeur apprend des catégories documentaires au-dessus du hasard uniforme sur ce petit échantillon, mais avec une erreur de 6 documents sur 16. Troisièmement, des interventions sur l'état caché altèrent la prédiction, ce qui permet une première étude causale à l'échelle des coordonnées.

Elles ne soutiennent pas que le modèle comprend le sens des catégories, sait suivre une instruction, raisonne de manière générale, est aligné conversationnellement ou reproduit les mécanismes d'un LLM enseignant. Il s'agit d'un tiny language model causal réel mais de faible capacité, pas d'un modèle compétitif avec un LLM moderne.

8 Conséquences théoriques, conceptuelles et opérationnelles

8.1 Théoriques

Une architecture à état récurrent partagé peut être entraînée avec un objectif de prédiction du prochain octet et un signal auxiliaire de facette. Le coût de l'ablation indique que les coordonnées n'ont pas toutes la même influence sous ce protocole.

8.2 Conceptuelles

Une « facette » est ici une définition opérationnelle du dataset (type documentaire). Les experts ne constituent pas automatiquement des mécanismes sémantiques indépendants. Une classification correcte ne doit pas être confondue avec la découverte causale d'une organisation cognitive.

8.3 Opérationnelles

Le produit possède un package autonome et peut être chargé depuis son dossier ou le ZIP depuis le registre de modèles local. Le backend Web a été démarré et testé en boucle locale : `/api/health`, chargement via `/api/models/load`, puis génération via `/api/chat`. L'interface sélectionne un modèle chargé ; le modèle peut donc être utilisé dans l'UI de chat comme source active.

9 Audit des restrictions de code

L'inspection du chemin v3 a trouvé plusieurs choix susceptibles d'abaisser la capacité observée sans preuve qu'ils aient été intentionnels : (i) l'ancien entraînement travaillait sur un extrait de corpus et une séparation séquentielle ; (ii) des arguments de configuration tels que `heads` ou `ff` ne correspondaient pas à des sous-modules utilisés par la GRU ; (iii) la génération pouvait renvoyer le contexte complet à chaque étape tout en gardant l'état récurrent, ce qui comptait deux fois le passé ; (iv) le chemin chat comportait une fenêtre glissante implicite.

Un audit complémentaire du registre local a trouvé deux autres défauts : la découverte de modèles pouvait afficher le dossier parent à la place d'une archive ZIP, et la protection contre la traversée de chemin dans les ZIP comparait de simples préfixes de chaînes. La découverte référence désormais l'archive elle-même ; l'extraction vérifie que chaque chemin résolu reste un descendant strict du répertoire cible. Les dossiers Hugging Face locaux avec poids ‘model.safetensors‘ ou index shardés sont également reconnus, si les dépendances optionnelles sont installées. Ces corrections ont leurs tests.

Les corrections v3_0 ciblent ces limites observables dans le chemin multi-facettes et le chargement local. L'ancien chemin d'entraînement est gardé pour compatibilité, sans être promu au statut de pipeline officiel de cette expérience. Aucun élément ne démontre une volonté de sabotage ; l'analyse est limitée aux comportements établis du code. Il serait également incorrect de considérer toute taille ou toute faiblesse de performance comme une restriction artificielle : une partie vient tout simplement des 54 536 paramètres, du corpus et de l'objectif d'entraînement.

10 Limites et contre-exemples

— Corpus artificiel, contrôlé, original et de taille modeste ; les résultats ne se généralisent pas automatiquement au texte naturel.

— Test de 16 documents et seulement deux documents par catégorie ; les incertitudes sont importantes.

— Représentation byte-level qui apprend les mots via des séquences longues d'octets ; aucun vocabulaire sous-mots.
— Modèle GRU une couche et 54 536 paramètres ; les générations longues restent faibles malgré l'amélioration de la NLL.
— Les tentatives pilotes avec routeur ont été faibles (25% de précision sur un jeu de développement). Elles ont été conservées comme résultats défavorables et ont mené à corriger l'objectif et à constituer un test final frais.
— La confusion de certaines classes persiste dans le test final ; 62.5% n'est pas une preuve de généralisation robuste.
— Les résultats ne prouvent pas une équivalence à SmolLM2, TinyLLM ou à un LLM de taille supérieure.
— Le fichier BAT Windows a été validé statiquement dans l'environnement Linux, mais n'a pas été exécuté physiquement sous Windows (ni via Wine, absent de l'environnement).

11 Conclusions

L'expérience atteint son objectif d'entraînement : NGeGenBrain v3_0 produit un modèle causal byte-level de langage réellement entraîné, autonome et chargeable, avec amélioration mesurée sur le test anglais contrôlé. La modification multi-facettes apporte un objectif de supervision additionnel, des mesures par facette et une première expérience d'intervention. Le modèle doit être décrit exactement comme « tiny causal language model expérimental autonome ».

La réussite n'est ni artificiellement garantie ni révoquée : elle est définie par les résultats constatés. Le modèle n'est pas encore un assistant de qualité et la méthode ne démontre pas avoir reconstruit les mécanismes internes d'un LLM externe. Ces questions demeurent des hypothèses de recherche à tester par des ensembles plus larges, multi-graines, des corpus naturels de référence, des essais adversariaux, des tests de correction et des interventions mécanistes plus riches.

12 Statut du modèle

Validé Modèle autonome entraîné ; chargement et sauvegarde des poids ; génération byte-level ; chargement de dossier/ZIP ; routeur et experts multi-facettes ; 60 tests logiciels passés ; smoke test de l'API Web.

Mesuré avec limites Amélioration de la NLL/test ; baseline trigramme dépassée sur ce corpus ; classification 10/16 ; effets d'ablation observés.

Non démontré Compétence conversationnelle générale, cohérence longue, raisonnement général, compréhension sémantique, équivalence mécaniste à un LLM externe, transfert à des corpus naturels larges.

13 Traçabilité

Artefact	Trace / empreinte
Dataset	`research/english_dataset_v3_0_final2/dataset_manifest.json` ; SHA-256 `d9919e0713bd33863dd08b30dd2b5d7974da6892e2bb2da2f6485877ca0b9281`.

Poids v3_0	`model_produit_NG_v3_0/model/model.pt` ; SHA-256 `8227 4b721bf87f307f9738003ff706f696a0e47972c13cc2afbe5 cf832c29743`.
Poids v3 de base	SHA-256 `d12a7d1e4cb3cb3d6b72ba9df5416fcea0f3b9743 998743130b7c6643130aa83`.
Résultats agrégés	`research/experiments_v3_0/run_final2_20261028/experiment_summa`
Baseline	`.../baseline_observation.json`.
Observations multi-facettes	`.../multifacet_observation.json`.
Audit code	`audit/rapport_audit_code_NGeGenBrain_v3_0.md`.
Tests	`audit/test_final.log` ; compte rendu détaillé dans le bundle audit.
API/registre	`audit/registry_model_load_chat.json` et `audit/web_api_smoke_result.json`.

14 Reproductibilité

Configuration du run final : graine 20261028, 900 étapes, batch 12, longueur des séquences 96 octets, learning rate 0.001, poids de la facette 0.8, rang adaptateurs 4, CPU, PyTorch 2.10.0+cpu. Le checkpoint choisi est la dernière étape (900) parce qu'elle avait la plus faible perte conjointe de validation. Le test final était réservé jusqu'au gel du checkpoint. Le modèle livré permet l'inférence sans dépendre des fichiers intermédiaires de recherche, tandis que les journaux et checkpoints de développement sont conservés dans le bundle de suivi.